SIMATS ENGINEERING

ASSESSMENT TOOL 1: Expert System Development Project

COURSE NAME: ARTIFICIAL INTELLIGENCE AND EXPERT SYSTEMS **COURSE CODE: MLA010**

COURSE OUTCOME COVERED

CO 5: *Construct rule-based expert systems using production rules, forward and backward chaining, and by distinguishing procedural and non-procedural paradigms across development stages.*(BTL 3)

Assessment Tool Weightage: 40%

Total Marks: 100 **Time: 90 Mins**

No. of Questions: 1

Annexure A

Expert System Development Project

Code of Conduct:

I P Surendra Kumar Reddy (192425224) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: P Surendra Kumar Reddy

Criterion	Weightage	Marks Obtained
1. Problem Analysis and Domain Understanding	10%	
2. Knowledge Acquisition and Knowledge Representation	10%	
3. Production Rule / Knowledge Base Design	15%	
4. Prolog Implementation and Programming	15%	
5. Forward Chaining Implementation and Demonstration	10%	
6. Backward Chaining Implementation and Demonstration	10%	
7. Procedural and Non-Procedural Paradigm Analysis	10%	
8. Unification, Backtracking and Inference Accuracy	10%	
9. Testing, Results and System Demonstration	5%	
10. Documentation, GitHub Repository and Technical Presentation	5%	
Total	**100%**	

Signature of the Faculty Total Marks Awarded

ANSWER ANY ONE OF THE FOLLOWING

Tool: SWI-Prolog

Usage of Prolog: Students shall use **Prolog** to develop a rule-based expert system by representing domain knowledge as **facts and production rules**, and implementing reasoning through **forward and backward chaining**. The project should demonstrate Prolog's **declarative/non-procedural nature**, logical inference, rule matching, and automated conclusion generation.

S.No.	Scenario-Based Expert System Development Question
1	**Medical Diagnosis Expert System:** Develop a rule-based expert system that identifies possible diseases based on symptoms, patient observations, and basic clinical information. Represent domain knowledge using production rules and implement both forward and backward chaining to derive conclusions.
2	**Crop Disease Advisory System:** Develop an expert system that identifies possible crop diseases based on leaf symptoms, soil conditions, weather conditions, and visible plant characteristics. Construct suitable production rules and demonstrate diagnosis using forward and backward chaining.
3	**Automobile Fault Diagnosis System:** Design an expert system that identifies probable vehicle faults based on symptoms such as engine noise, starting problems, warning indicators, abnormal vibration, and reduced mileage. Implement the knowledge base using production rules and compare forward and backward reasoning.
4	**Student Academic Advisory System:** Develop an expert system that recommends suitable academic interventions based on attendance, internal marks, assignment performance, learning difficulties, and previous academic performance. Use production rules and demonstrate both forward and backward chaining.
5	**Cybersecurity Threat Diagnosis System:** Construct a rule-based expert system that identifies possible cybersecurity threats from events such as repeated login failures, unusual network traffic, suspicious file activity, and unauthorized access attempts. Implement production rules and demonstrate how forward and backward chaining arrive at a threat classification.

Report Format:

Stage	Student Activity	Expected Output
1. Domain Analysis	Identify the problem domain, entities, facts, symptoms/conditions, and possible conclusions.	Problem definition and domain model
2. Knowledge Acquisition	Collect domain knowledge from reliable sources or domain experts.	Knowledge specification
3. Knowledge Representation	Convert domain knowledge into IF–THEN production rules.	Knowledge base
4. Inference Design	Implement forward chaining and backward chaining.	Inference engine
5. Paradigm Analysis	Distinguish how the solution can be expressed using procedural and non-procedural approaches.	Paradigm comparison
6. System Development	Integrate the knowledge base, inference engine, user interface, and explanation facility.	Working expert system
7. Testing	Test the system using multiple facts and scenarios.	Test cases and outputs
8. Evaluation	Analyze correctness, coverage, consistency, and reasoning performance.	Evaluation results
9. Documentation	Prepare technical documentation and GitHub repository.	Project report + GitHub
10. Demonstration	Demonstrate the system and explain the reasoning process.	Project presentation/viva

STUDENT ACADEMIC ADVISOR EXPERT SYSTEM USING PROLOG

1. Introduction

An expert system is an Artificial Intelligence system that uses a knowledge base and inference mechanisms to solve problems that normally require human expertise. This project develops a **Student Academic Advisor Expert System** using Prolog.

The system analyzes student academic information such as attendance, internal marks, assignment performance, and learning difficulty. Based on these facts, the system applies production rules to derive conclusions such as academic risk, attendance improvement, extra learning support, assignment support, and faculty counselling.

The system demonstrates two important reasoning approaches:

- Forward chaining
- Backward chaining

The project also demonstrates Prolog concepts including facts, rules, variables, unification, backtracking, and logical inference.

2. Problem Statement

Students may experience academic difficulties because of low attendance, poor internal marks, poor assignment performance, or learning difficulties. Manually identifying students who require academic support can be time-consuming.

The objective of this project is to develop an intelligent academic advisory system that analyzes student information and automatically provides suitable academic recommendations.

The system accepts student information as facts and uses predefined production rules to generate conclusions and recommendations.

Inputs

The main student attributes considered by the system are:

- Student name
- Attendance level
- Internal marks

- Assignment performance
- Learning difficulty

Outputs

The system produces academic conclusions and recommendations such as:

- Academic Risk
- Improve Attendance
- Extra Learning Support
- Assignment Support
- Faculty Counselling
- Student Requires Academic Attention
- Student Performing Well

3. Objectives

The main objectives of the project are:

1. To develop a rule-based expert system using Prolog.
2. To represent student academic information using facts.
3. To represent academic decision-making knowledge using production rules.
4. To implement forward chaining for data-driven reasoning.
5. To implement backward chaining for goal-driven reasoning.
6. To demonstrate unification and backtracking in Prolog.
7. To generate meaningful academic recommendations.
8. To test the system using multiple student scenarios.
9. To provide understandable reasoning traces showing why recommendations were generated.

4. Domain Analysis

The selected domain is **student academic advising**.

The expert system considers several academic indicators.

Attribute	Possible Values	Meaning
Attendance	high / low	Indicates student's attendance level
Internal Marks	high / low	Indicates performance in internal assessments
Assignment Performance	good / poor	Indicates assignment quality
Learning Difficulty	yes / no	Indicates whether additional learning support may be required

The system combines these attributes through logical rules.

For example, when a student has low attendance and low internal marks, the system derives the conclusion that the student is at academic risk.

5. Knowledge Acquisition

The knowledge represented in the system consists of academic advising conditions and their corresponding recommendations.

The knowledge was converted into Prolog facts and rules.

For example, a student's academic information can be represented as:

student(suri, low, low, poor, yes).

This fact represents:

- Student: suri
- Attendance: low
- Internal marks: low
- Assignment performance: poor
- Learning difficulty: yes

The expert knowledge is then represented using production rules.

6. Knowledge Representation

Prolog represents the knowledge using facts, predicates, variables, and rules.

6.1 Student Facts

A student fact follows the structure:

student(Name, Attendance, Marks, Assignment, Difficulty).

Example:

student(suri, low, low, poor, yes).

Other students are also represented in the knowledge base to test different academic situations.

6.2 Production Rules

The expert system uses rules to derive conclusions from student facts.

Examples of the implemented reasoning are:

```
academic_risk(Name) :-
   student(Name, low, low, _, _).
```

Another academic-risk condition is represented using low attendance and poor assignment performance.

```
academic_risk(Name) :-
   student(Name, low, _, poor, _).
```

A low attendance condition produces an attendance recommendation:

```
attendance_improvement(Name) :-
   student(Name, low, _, _, _).
```

A learning difficulty produces extra learning support:

```
extra_learning_support(Name) :-
   student(Name, _, _, _, yes).
```

Poor assignment performance produces assignment support:

```
assignment_support(Name) :-
   student(Name, _, _, poor, _).
```

Academic risk produces faculty counselling:

```
faculty_counselling(Name) :-
    academic_risk(Name).
```

A student with high attendance, high marks, and good assignments can be classified as having overall good performance.

7. Inference Design

The expert system uses two reasoning strategies:

7.1 Forward Chaining

Forward chaining is a data-driven reasoning approach.

The system starts with the known student facts and checks which rules can be applied. When the conditions of a rule are satisfied, the corresponding conclusion is derived.

For student suri, the initial facts are:

Attendance: low
Internal Marks: low
Assignment Performance: poor
Learning Difficulty: yes

The system fires applicable rules and derives:

academic_risk
attendance_improvement
extra_learning_support
assignment_support
faculty_counselling

The final recommendations are generated from these conclusions.

Forward Chaining Execution

```
102 ?-  forward_chaining(suri).

========================================
          FORWARD CHAINING
========================================
Initial Facts
Student: suri
Attendance: low
Internal Marks: low
Assignment Performance: poor
Learning Difficulty: yes

Rule 1 Fired:
Low attendance + Low internal marks
=> Academic Risk

Rule 3 Fired:
Low attendance
=> Improve Attendance

Rule 4 Fired:
Learning difficulty detected
=> Extra Learning Support

Rule 5 Fired:
Poor assignment performance
=> Assignment Support

Rule 6 Fired:
Academic risk detected
=> Faculty Counselling

----------------------------------------
Derived Conclusions
----------------------------------------
-> academic_risk
-> attendance_improvement
-> extra_learning_support
-> assignment_support
-> faculty_counselling
true.
```

Figure 1: Forward Chaining Execution for Suri

The screenshot demonstrates the initial facts, fired rules, and derived conclusions.

8. Backward Chaining

Backward chaining is a goal-driven reasoning approach.

Instead of starting from all available facts, the system begins with a goal and checks whether that goal can be proven from the available facts and rules.

For example, the system checks goals such as:

academic_risk
attendance_improvement
extra_learning_support
assignment_support
faculty_counselling
overall_good_performance

For suri, the system successfully proves the academic-risk-related goals and does not prove the overall-good-performance goal.

Backward Chaining Execution

```
103 ?-  backward_chaining(suri).

========================================
          BACKWARD CHAINING
========================================
Goal-driven reasoning:

Goal: academic_risk -> TRUE
Goal: attendance_improvement -> TRUE
Goal: extra_learning_support -> TRUE
Goal: assignment_support -> TRUE
Goal: faculty_counselling -> TRUE
Goal: overall_good_performance -> FALSE

Backward chaining completed.
true.
```

Figure 2: Backward Chaining Execution for Suri

The output demonstrates goal-driven reasoning because each goal is evaluated as TRUE or FALSE.

9. Unification and Backtracking

Unification is an important Prolog mechanism that allows variables to match values.

The following query was used:

?- academic_risk(X).

The system returns students satisfying the academic-risk rule, for example:

X = rahul ;
X = suri ;
X = rahul ;
X = arun ;
X = suri.

The exact order and repeated results depend on the rules present in the knowledge base.

The semicolon allows Prolog to search for additional solutions through backtracking.

The query:

?- academic_risk(suri).

returns:

true.

This demonstrates that suri satisfies the academic-risk condition.

Unification and Backtracking Evidence

```
104 ?-  academic_risk(suri).
true .

105 ?-  academic_risk(X).
X = rahul ;
X = suri ;
X = rahul ;
X = arun ;
X = suri.
```

Figure 3: Unification and Backtracking

10. Procedural and Non-Procedural Paradigm Analysis

Procedural Approach

A procedural programming approach describes **how** a problem should be solved. The programmer normally specifies the sequence of operations and control flow.

For example, a procedural implementation could explicitly perform the following steps:

1. Read student information.
2. Check attendance.
3. Check internal marks.
4. Check assignments.
5. Check learning difficulty.
6. Generate recommendations.

The programmer controls the order of execution.

Non-Procedural / Declarative Approach

Prolog follows a declarative programming style. The programmer mainly describes **what is true** using facts and rules.

For example:

academic_risk(Name) :-

student(Name, low, low, _, _).

The rule describes the relationship between the student's facts and the conclusion. Prolog determines how to search for a solution using its inference mechanism.

Comparison

Feature	Procedural	Prolog / Declarative
Main focus	How to solve	What is true
Control flow	Explicit	Mostly handled by Prolog
Knowledge representation	Procedures	Facts and rules
Reasoning	Programmer-defined sequence	Logical inference
Backtracking	Usually implemented manually	Built into Prolog
Unification	Not normally central	Core Prolog mechanism
Suitability for expert systems	More implementation effort	Highly suitable

For this project, the declarative approach is suitable because academic knowledge can naturally be expressed as facts and rules.

11. Prolog Implementation

The system was implemented using Prolog and executed using the Prolog interpreter.

The major components are:

Knowledge Base

Contains student facts.

Production Rules

Determine academic conclusions from the student facts.

Forward Chaining Module

Starts with initial facts and derives applicable conclusions.

Backward Chaining Module

Checks predefined goals using goal-driven reasoning.

Advisory Module

Combines the reasoning results and displays the final recommendations.

Testing Module

Runs multiple student scenarios to demonstrate the behavior of the system.

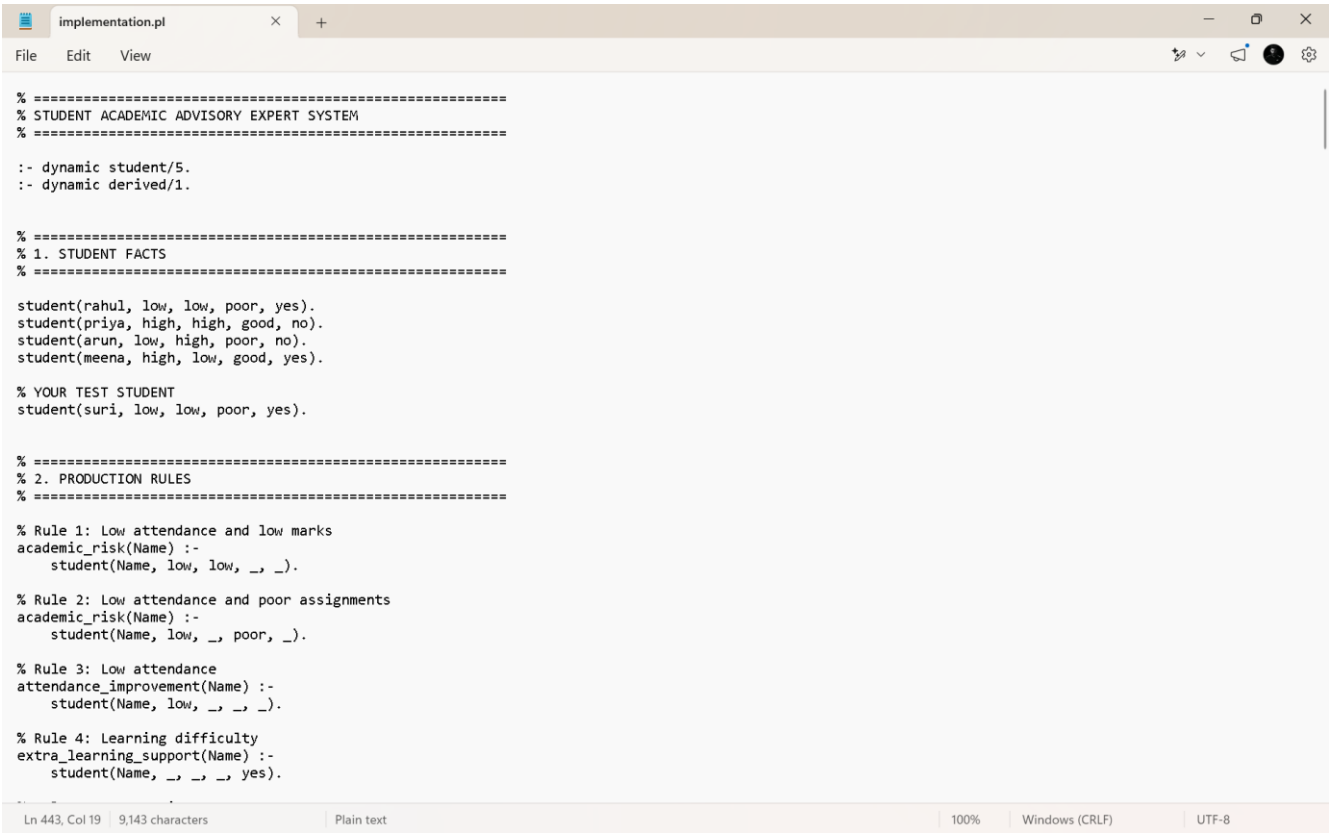

Figure 4: Prolog Source Code

12. System Architecture

The overall operation of the system can be represented as:

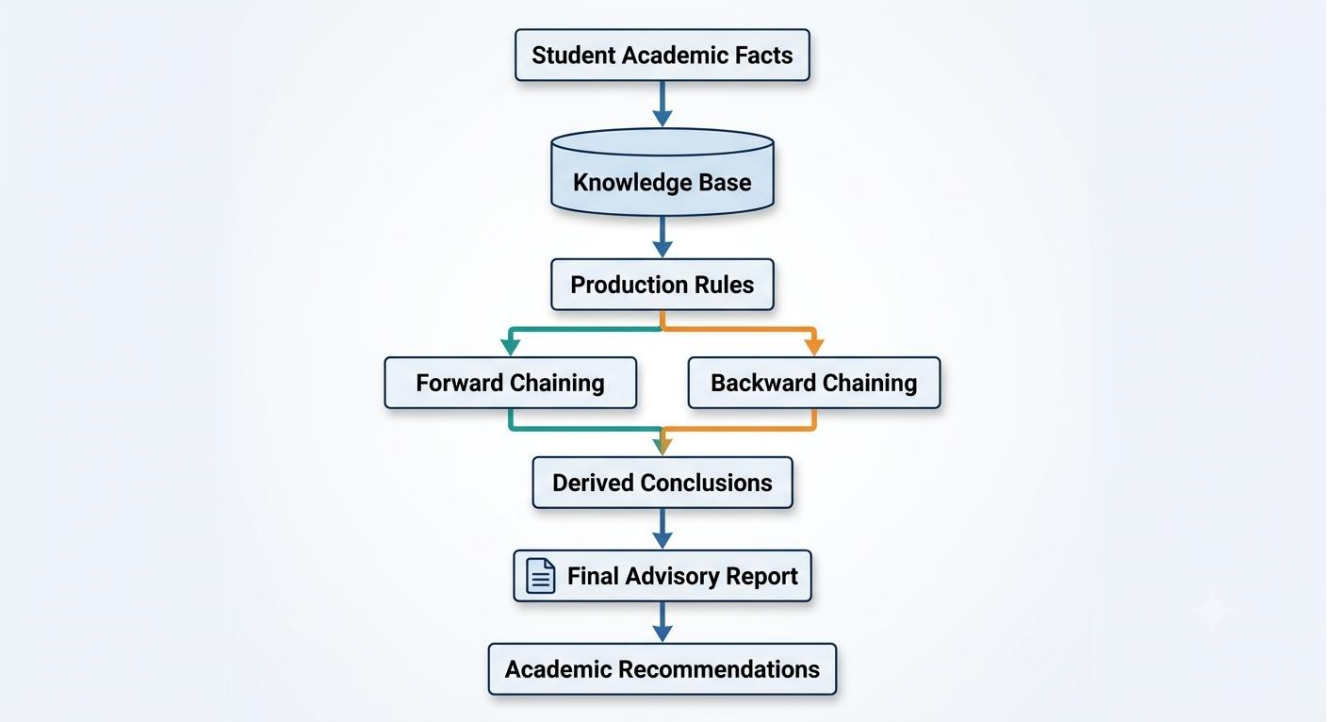

Figure 5: System Architecture

system therefore follows a knowledge-based reasoning architecture in which facts are processed using rules to generate conclusions.

13. Algorithm

Forward Chaining Algorithm

1. Accept the student's facts.
2. Display the initial facts.
3. Check each production rule.
4. Determine whether the rule conditions are satisfied.
5. Fire the applicable rule.
6. Add the derived conclusion.
7. Continue until all applicable rules have been processed.
8. Display the derived conclusions.
9. Generate recommendations.

Backward Chaining Algorithm

1. Select an advisory goal.
2. Search for a rule that can establish the goal.
3. Check the conditions of that rule.
4. Match the rule conditions with known facts.
5. If the conditions are satisfied, the goal becomes TRUE.
6. Otherwise, the goal becomes FALSE.
7. Repeat for the required goals.
8. Display the reasoning results.

14. Testing and Experimental Results

The system was tested using multiple student scenarios.

The test cases included:

- Rahul
- Priya
- Arun
- Meena
- Suri

Different combinations of attendance, marks, assignment performance, and learning difficulty were used.

Test Case Summary

Student	Main Academic Situation	Result
Rahul	Low attendance, low marks, poor assignments, learning difficulty	Academic risk and multiple support recommendations
Priya	High attendance, high marks, good assignments, no learning difficulty	Overall good performance
Arun	Low attendance, high marks, poor assignments	Academic risk and attendance/assignment support
Meena	High attendance, low marks, good assignments, learning difficulty	Extra learning support
Suri	Low attendance, low marks, poor assignments, learning difficulty	Academic risk and multiple recommendations

The test output demonstrates that different combinations of facts lead to different conclusions.

15. Test Case: Suri

The most important demonstration is the advisory query:

?- advisor(suri).

The system displays:

Student: suri

Attendance: low
Internal Marks: low
Assignment Performance: poor
Learning Difficulty: yes

The forward chaining process derives:

academic_risk
attendance_improvement
extra_learning_support
assignment_support
faculty_counselling

The backward chaining process confirms the corresponding goals.

The final advisory report provides:

[1] Improve Attendance
[2] Provide Extra Learning Support
[3] Improve Assignment Performance
[4] Faculty Counselling Recommended
[5] Student Requires Academic Attention

```
106 ?-  advisor(suri).

========================================
      STUDENT ACADEMIC ADVISOR
========================================
Student: suri

========================================
          FORWARD CHAINING
========================================
Initial Facts
Student: suri
Attendance: low
Internal Marks: low
Assignment Performance: poor
Learning Difficulty: yes

Rule 1 Fired:
Low attendance + Low internal marks
=> Academic Risk

Rule 3 Fired:
Low attendance
=> Improve Attendance

Rule 4 Fired:
Learning difficulty detected
=> Extra Learning Support

Rule 5 Fired:
Poor assignment performance
=> Assignment Support

Rule 6 Fired:
Academic risk detected
=> Faculty Counselling

----------------------------------------
Derived Conclusions
----------------------------------------
-> academic_risk
-> attendance_improvement
-> extra_learning_support
-> assignment_support
-> faculty_counselling
```

```
========================================
          BACKWARD CHAINING
========================================
Goal-driven reasoning:

Goal: academic_risk -> TRUE
Goal: attendance_improvement -> TRUE
Goal: extra_learning_support -> TRUE
Goal: assignment_support -> TRUE
Goal: faculty_counselling -> TRUE
Goal: overall_good_performance -> FALSE

Backward chaining completed.

========================================
          FINAL ADVISORY REPORT
========================================

Recommendations:

[1] Improve Attendance
[2] Provide Extra Learning Support
[3] Improve Assignment Performance
[4] Faculty Counselling Recommended
[5] Student Requires Academic Attention
true .
```

Figure 6: Final Advisory Report for Suri

16. Results and Analysis

The experimental results show that the expert system successfully applies its production rules to different student scenarios.

For Suri, multiple academic difficulties are detected simultaneously. Low attendance and low internal marks lead to the academic-risk conclusion. Low attendance generates an attendance-improvement

recommendation. Learning difficulty generates extra learning support. Poor assignment performance generates assignment support. Academic risk subsequently results in a faculty-counselling recommendation.

The system also demonstrates the opposite type of situation. Priya has high attendance, high internal marks, good assignments, and no learning difficulty. The system therefore derives the overall-good-performance conclusion and produces a positive advisory result.

Meena demonstrates that a student does not necessarily need to satisfy the academic-risk rules to receive support. Her learning difficulty causes the system to recommend extra learning support.

These results demonstrate that the rule-based system can distinguish between different academic situations based on the supplied facts.

17. Unification and Backtracking Results

The query:

?- academic_risk(suri).

returned:

true.

This confirms that the student satisfies an academic-risk rule.

The query:

?- academic_risk(X).

returned multiple matching students.

This demonstrates:

- Variable matching
- Unification
- Multiple solutions
- Prolog backtracking

Therefore, the implementation demonstrates important logical programming features required for the expert system.

18. Limitations

The current system has several limitations:

1. The knowledge base contains a limited number of student attributes.
2. The academic categories are represented using simple symbolic values such as high, low, good, and poor.
3. The recommendations depend on predefined rules.
4. The system does not learn new rules automatically from historical student data.
5. The system does not use numerical academic scores directly.
6. The quality of the recommendations depends on the correctness and completeness of the knowledge base.
7. The current system is a prototype and should support, rather than replace, human academic advising.

19. Future Enhancements

The system can be enhanced in several ways:

1. Add more student attributes such as study hours and previous examination performance.
2. Include numerical marks and attendance percentages.
3. Add more academic rules.
4. Add confidence levels to recommendations.
5. Develop a graphical or web-based interface.
6. Store student information in a database.
7. Add explanation facilities showing exactly why each recommendation was produced.
8. Allow academic advisors to update the knowledge base.
9. Add more test cases.
10. Integrate historical academic data to improve the advisory process.

20. Conclusion

The Student Academic Advisor Expert System demonstrates how Artificial Intelligence and knowledge-based reasoning can be applied to academic advising.

The system represents student information using Prolog facts and uses production rules to derive academic conclusions. Forward chaining demonstrates data-driven reasoning from initial facts to conclusions, while backward chaining demonstrates goal-driven reasoning.

The project also demonstrates important Prolog concepts such as variables, unification, backtracking, and logical inference.

Testing with Rahul, Priya, Arun, Meena, and Suri shows that the system produces different recommendations according to the academic conditions of each student.

For Suri, the system successfully identifies academic risk and recommends improvement in attendance, additional learning support, assignment support, and faculty counselling.

Overall, the project demonstrates a functional rule-based expert system capable of providing understandable academic recommendations through logical reasoning.

21. Screenshots / Evidence

The following screenshots provide evidence of the implemented expert system and its testing results.

```
102 ?-  forward_chaining(suri).

========================================
          FORWARD CHAINING
========================================
Initial Facts
Student: suri
Attendance: low
Internal Marks: low
Assignment Performance: poor
Learning Difficulty: yes

Rule 1 Fired:
Low attendance + Low internal marks
=> Academic Risk

Rule 3 Fired:
Low attendance
=> Improve Attendance

Rule 4 Fired:
Learning difficulty detected
=> Extra Learning Support

Rule 5 Fired:
Poor assignment performance
=> Assignment Support

Rule 6 Fired:
Academic risk detected
=> Faculty Counselling

----------------------------------------
Derived Conclusions
----------------------------------------
-> academic_risk
-> attendance_improvement
-> extra_learning_support
-> assignment_support
-> faculty_counselling
true.
```

Figure 1 – Forward Chaining

Query:

?- forward_chaining(suri).

Shows:

- Initial facts
- Rules fired
- Derived conclusions

```
103 ?-  backward_chaining(suri).

========================================
          BACKWARD CHAINING
========================================
Goal-driven reasoning:

Goal: academic_risk -> TRUE
Goal: attendance_improvement -> TRUE
Goal: extra_learning_support -> TRUE
Goal: assignment_support -> TRUE
Goal: faculty_counselling -> TRUE
Goal: overall_good_performance -> FALSE

Backward chaining completed.
true.
```

Figure 2 – Backward Chaining

Query:

?- backward_chaining(suri).

Shows:

- Goal-driven reasoning
- TRUE/FALSE goals
- Completion of backward chaining

```
104 ?-  academic_risk(suri).
true .

105 ?-  academic_risk(X).
X = rahul ;
X = suri ;
X = rahul ;
X = arun ;
X = suri.
```

Figure 3 – Unification and Backtracking

Queries:

?- academic_risk(suri).

?- academic_risk(X).

Shows:

- true
- Multiple values of X
- Backtracking

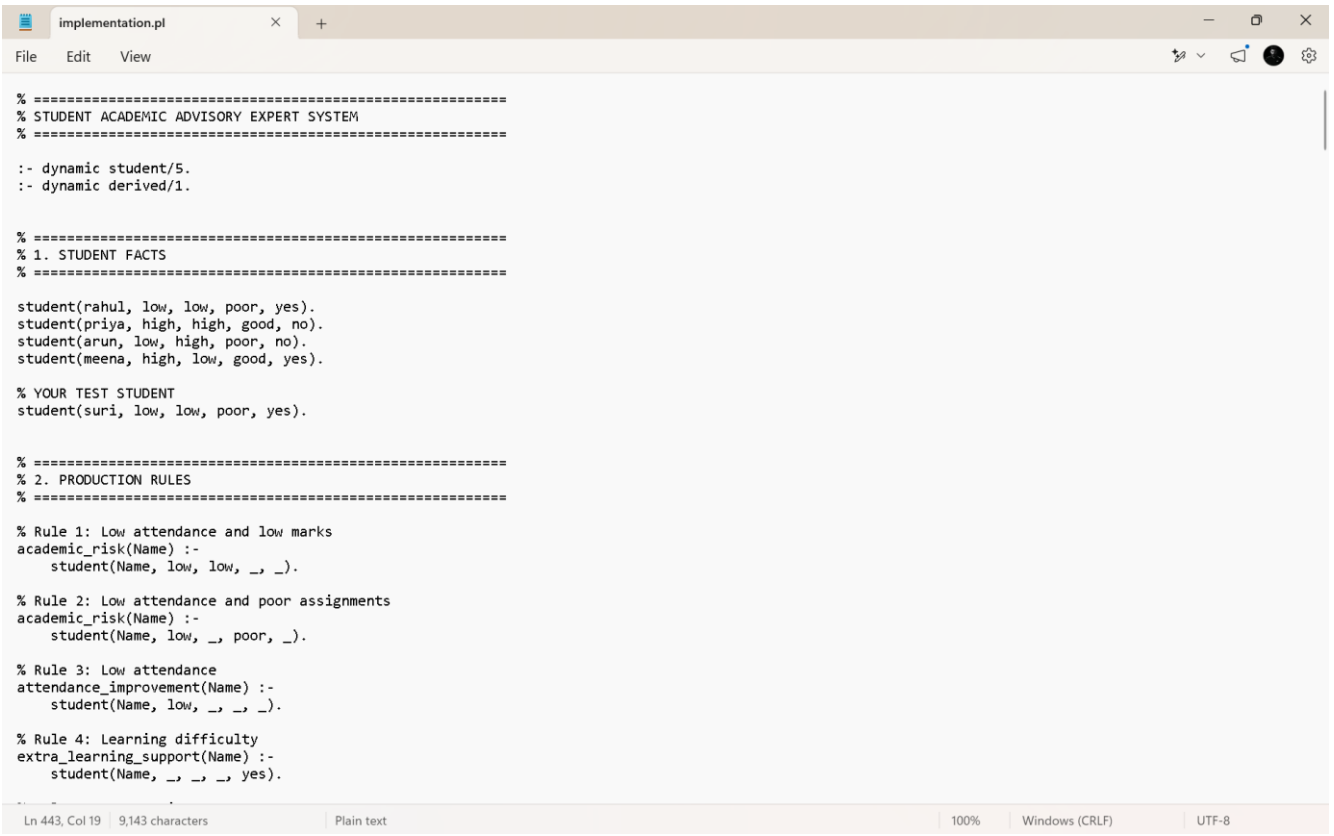

Figure 4 – Prolog Source Code

Shows:

- Student facts
- Production rules
- Forward chaining predicates
- Backward chaining predicates
- Advisory predicate

```
106 ?-  advisor(suri).

========================================
      STUDENT ACADEMIC ADVISOR
========================================
Student: suri

========================================
          FORWARD CHAINING
========================================
Initial Facts
Student: suri
Attendance: low
Internal Marks: low
Assignment Performance: poor
Learning Difficulty: yes

Rule 1 Fired:
Low attendance + Low internal marks
=> Academic Risk

Rule 3 Fired:
Low attendance
=> Improve Attendance

Rule 4 Fired:
Learning difficulty detected
=> Extra Learning Support

Rule 5 Fired:
Poor assignment performance
=> Assignment Support

Rule 6 Fired:
Academic risk detected
=> Faculty Counselling

----------------------------------------
Derived Conclusions
----------------------------------------
-> academic_risk
-> attendance_improvement
-> extra_learning_support
-> assignment_support
-> faculty_counselling


========================================
          BACKWARD CHAINING
========================================
Goal-driven reasoning:

Goal: academic_risk -> TRUE
Goal: attendance_improvement -> TRUE
Goal: extra_learning_support -> TRUE
Goal: assignment_support -> TRUE
Goal: faculty_counselling -> TRUE
Goal: overall_good_performance -> FALSE

Backward chaining completed.

========================================
          FINAL ADVISORY REPORT
========================================

Recommendations:

[1] Improve Attendance
[2] Provide Extra Learning Support
[3] Improve Assignment Performance
[4] Faculty Counselling Recommended
[5] Student Requires Academic Attention
true .
```

Figure 5 – Final Advisory System

Query:

?- advisor(suri).

Shows:

- Student information
- Forward chaining

- Backward chaining
- Final recommendations

Test Cases:

Test Case 1: Rahul

```
102 ?- test.

========================================
              TEST CASES
========================================

========== TEST CASE 1: RAHUL ==========

========================================
      STUDENT ACADEMIC ADVISOR
========================================
Student: rahul

========================================
         FORWARD CHAINING
========================================
Initial Facts
Student: rahul
Attendance: low
Internal Marks: low
Assignment Performance: poor
Learning Difficulty: yes

Rule 1 Fired:
Low attendance + Low internal marks
=> Academic Risk

Rule 3 Fired:
Low attendance
=> Improve Attendance

Rule 4 Fired:
Learning difficulty detected
=> Extra Learning Support

Rule 5 Fired:
Poor assignment performance
=> Assignment Support

Rule 6 Fired:
Academic risk detected
=> Faculty Counselling

----------------------------------------
Derived Conclusions
----------------------------------------
-> academic_risk
-> attendance_improvement
-> extra_learning_support
-> assignment_support
-> faculty_counselling
```

Test Case 2: Priya

```
========================================
         BACKWARD CHAINING
========================================
Goal-driven reasoning:

Goal: academic_risk -> TRUE
Goal: attendance_improvement -> TRUE
Goal: extra_learning_support -> TRUE
Goal: assignment_support -> TRUE
Goal: faculty_counselling -> TRUE
Goal: overall_good_performance -> FALSE

Backward chaining completed.

========================================
          FINAL ADVISORY REPORT
========================================

Recommendations:

[1] Improve Attendance
[2] Provide Extra Learning Support
[3] Improve Assignment Performance
[4] Faculty Counselling Recommended
[5] Student Requires Academic Attention

========== TEST CASE 2: PRIYA ==========

========================================
      STUDENT ACADEMIC ADVISOR
========================================
Student: priya

========================================
         FORWARD CHAINING
========================================
Initial Facts
Student: priya
Attendance: high
Internal Marks: high
Assignment Performance: good
Learning Difficulty: no

Rule 7 Fired:
High attendance + High marks + Good assignments
=> Overall Good Performance
```

Test Case 3: Arun

```
------------------------------------------
Derived Conclusions
------------------------------------------
-> overall_good_performance

==========================================
          BACKWARD CHAINING
==========================================
Goal-driven reasoning:

Goal: academic_risk -> FALSE
Goal: attendance_improvement -> FALSE
Goal: extra_learning_support -> FALSE
Goal: assignment_support -> FALSE
Goal: faculty_counselling -> FALSE
Goal: overall_good_performance -> TRUE

Backward chaining completed.

==========================================
          FINAL ADVISORY REPORT
==========================================

Recommendations:

[6] Student is Performing Well

========== TEST CASE 3: ARUN ==========

==========================================
      STUDENT ACADEMIC ADVISOR
==========================================
Student: arun

==========================================
          FORWARD CHAINING
==========================================
Initial Facts
Student: arun
Attendance: low
Internal Marks: high
Assignment Performance: poor
Learning Difficulty: no

Rule 2 Fired:
Low attendance + Poor assignments
=> Academic Risk

Rule 3 Fired:
Low attendance
=> Improve Attendance
```

Test Case 4: Meena

```
Rule 5 Fired:
Poor assignment performance
=> Assignment Support

Rule 6 Fired:
Academic risk detected
=> Faculty Counselling

------------------------------------------
Derived Conclusions
------------------------------------------
-> academic_risk
-> attendance_improvement
-> assignment_support
-> faculty_counselling

==========================================
          BACKWARD CHAINING
==========================================
Goal-driven reasoning:

Goal: academic_risk -> TRUE
Goal: attendance_improvement -> TRUE
Goal: extra_learning_support -> FALSE
Goal: assignment_support -> TRUE
Goal: faculty_counselling -> TRUE
Goal: overall_good_performance -> FALSE

Backward chaining completed.

==========================================
          FINAL ADVISORY REPORT
==========================================

Recommendations:

[1] Improve Attendance
[3] Improve Assignment Performance
[4] Faculty Counselling Recommended
[5] Student Requires Academic Attention

========== TEST CASE 4: MEENA ==========

==========================================
      STUDENT ACADEMIC ADVISOR
==========================================
Student: meena
```

Test Case 5: Suri

```
========================================
          FORWARD CHAINING
========================================
Initial Facts
Student: meena
Attendance: high
Internal Marks: low
Assignment Performance: good
Learning Difficulty: yes

Rule 4 Fired:
Learning difficulty detected
=> Extra Learning Support

----------------------------------------
Derived Conclusions
----------------------------------------
-> extra_learning_support

========================================
          BACKWARD CHAINING
========================================
Goal-driven reasoning:

Goal: academic_risk -> FALSE
Goal: attendance_improvement -> FALSE
Goal: extra_learning_support -> TRUE
Goal: assignment_support -> FALSE
Goal: faculty_counselling -> FALSE
Goal: overall_good_performance -> FALSE

Backward chaining completed.

========================================
          FINAL ADVISORY REPORT
========================================

Recommendations:

[2] Provide Extra Learning Support

========== TEST CASE 5: SURI ==========

========================================
      STUDENT ACADEMIC ADVISOR
========================================
Student: suri

==========================================
          FORWARD CHAINING
==========================================
Initial Facts
Student: suri
Attendance: low
Internal Marks: low
Assignment Performance: poor
Learning Difficulty: yes

Rule 1 Fired:
Low attendance + Low internal marks
=> Academic Risk

Rule 3 Fired:
Low attendance
=> Improve Attendance

Rule 4 Fired:
Learning difficulty detected
=> Extra Learning Support

Rule 5 Fired:
Poor assignment performance
=> Assignment Support

Rule 6 Fired:
Academic risk detected
=> Faculty Counselling

------------------------------------------
Derived Conclusions
------------------------------------------
-> academic_risk
-> attendance_improvement
-> extra_learning_support
-> assignment_support
-> faculty_counselling

==========================================
          BACKWARD CHAINING
==========================================
Goal-driven reasoning:

Goal: academic_risk -> TRUE
Goal: attendance_improvement -> TRUE
Goal: extra_learning_support -> TRUE
Goal: assignment_support -> TRUE
Goal: faculty_counselling -> TRUE
Goal: overall_good_performance -> FALSE
```

```
Backward chaining completed.

==========================================
          FINAL ADVISORY REPORT
==========================================

Recommendations:

[1] Improve Attendance
[2] Provide Extra Learning Support
[3] Improve Assignment Performance
[4] Faculty Counselling Recommended
[5] Student Requires Academic Attention
true .
```

Figure 7: Test Case Results – Rahul, Priya, Arun, Meena and Suri

Query:

?- test.

Shows the results for:

- Rahul
- Priya
- Arun
- Meena
- Suri

22. References

1. SWI-Prolog Documentation, Prolog language and predicates.
2. Sterling, L. and Shapiro, E., *The Art of Prolog: Advanced Programming Techniques*.
3. Bratko, I., *Prolog Programming for Artificial Intelligence*.
4. Russell, S. and Norvig, P., *Artificial Intelligence: A Modern Approach*.
5. Course materials on Artificial Intelligence and Expert Systems.

23. GitHub Repository

URL: https://github.com/suri307/MLA0103-AI/tree/main/CO5

EVALUATION RUBRICS:

Criterion	Weightage (%)	Excellent	Good	Average	Poor
1. Problem Analysis and Domain Understanding	10%	9–10% – Clearly analyzes the real-world problem, domain entities, facts, conditions, goals, and expected conclusions.	7–8% – Good understanding of the domain with minor omissions.	5–6% – Basic domain understanding with several missing elements.	0–4% – Poor or incorrect understanding of the problem domain.
2. Knowledge Acquisition and Representation	10%	9–10% – Acquires relevant domain knowledge and represents it accurately using well-structured Prolog facts and rules.	7–8% – Knowledge is appropriately represented with minor gaps.	5–6% – Basic knowledge representation with missing or unclear facts/rules.	0–4% – Inadequate or inappropriate knowledge representation.
3. Production Rule / Knowledge Base Design	15%	13–15% – Develops a comprehensive, consistent, and logically structured production-rule knowledge base with appropriate facts and rules.	10–12% – Well-designed knowledge base with minor inconsistencies or omissions.	7–9% – Basic rule base developed but several rules are incomplete or unclear.	0–6% – Knowledge base is incomplete, inconsistent, or inappropriate.
4. Prolog Implementation	15%	13–15% – Implements a fully functional expert system using Prolog facts, predicates, rules, variables, unification, and backtracking effectively.	10–12% – Functional Prolog implementation with minor technical issues.	7–9% – Basic implementation works with limited use of Prolog features.	0–6% – Implementation is incomplete, incorrect, or non-functional.

5. Forward Chaining	10%	9–10% – Correctly implements and demonstrates forward chaining from initial facts through applicable rules to final conclusions.	7–8% – Forward chaining works correctly with minor limitations.	5–6% – Basic forward reasoning demonstrated but with incomplete rule processing.	0–4% – Forward chaining is missing or incorrectly implemented.
6. Backward Chaining	10%	9–10% – Effectively demonstrates goal-driven backward chaining using Prolog's inference mechanism and clearly traces the reasoning process.	7–8% – Backward chaining works correctly with minor gaps.	5–6% – Basic backward reasoning demonstrated with limited explanation.	0–4% – Backward chaining is missing or incorrectly implemented.
7. Procedural and Non-Procedural Paradigm Analysis	10%	9–10% – Clearly distinguishes procedural and declarative/non-procedural approaches and demonstrates their application in the Prolog system.	7–8% – Provides a good distinction with minor conceptual gaps.	5–6% – Basic distinction is explained but lacks practical demonstration.	0–4% – Paradigms are misunderstood or not addressed.
8. Inference, Unification and Reasoning Accuracy	10%	9–10% – Produces accurate conclusions using unification, backtracking, and logical inference across diverse test cases.	7–8% – Reasoning is mostly accurate with minor errors.	5–6% – Basic inference works but produces occasional incorrect results.	0–4% – Inference is unreliable or incorrect.

9. Testing, Results and System Demonstration	5%	5% – Provides comprehensive test cases, accurate outputs, reasoning traces, and a convincing live demonstration.	4% – Good testing and demonstration with minor gaps.	2–3% – Limited test cases and basic demonstration.	0–1% – Testing or demonstration is missing/inadequate.
10. Documentation, GitHub and Technical Presentation	5%	5% – Complete documentation, well-organized GitHub repository, clear code, references, and professional presentation.	4% – Good documentation and repository with minor issues.	2–3% – Basic documentation/repository with several omissions.	0–1% – Poor documentation, missing repository, or inadequate presentation.
Total	100%				